Sounds of Seattle: Classifying Seattle Bird Calls

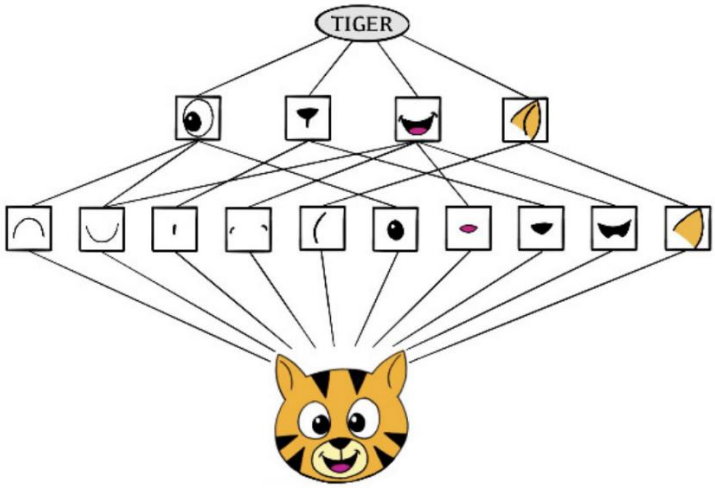
Tyler Franck

Introduction

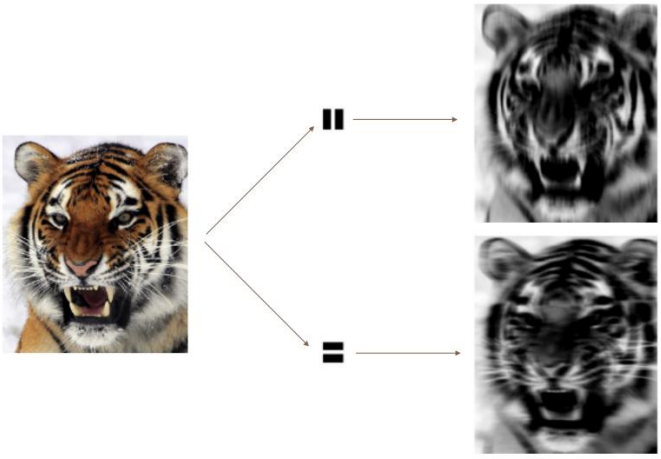
We will use deep learning to classify Seattle bird calls. The original data was taken from this Birdcall competition data (which was originally from the crowd-sourced wildlife sound archive Xeno-Canto, but we will be using a subsetted and processed version of the data (found here). The original data was subsetted to only include high quality data for 12 species of birds (see below), and it was processed to compute the *spectrograms* of the audio files. A spectrogram is a visual representation of sound frequency over time; you can think of it as an *image* of a sound. Importantly, this will allow us to classify bird calls using image classification techniques. In particular, we will be using *convolutional neural networks*. We will develop both a binary classification model to distinguish between the American Crow and Red-winged Blackbird, and a multi-class model to distinguish between all 12 species.

Theoretical Background

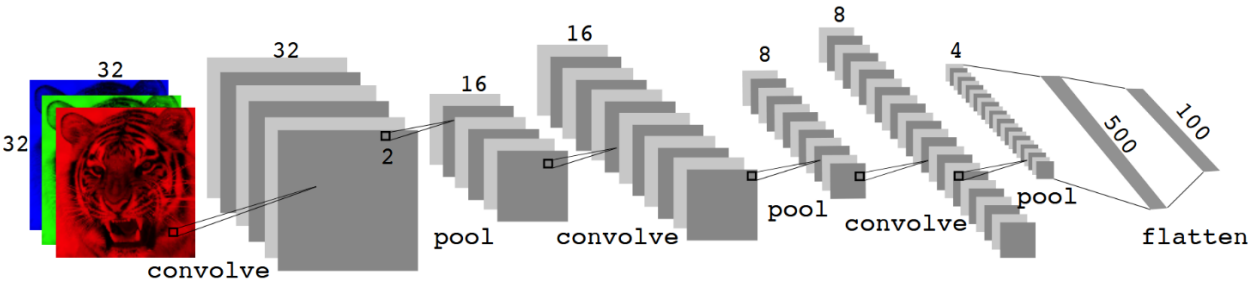
Neural networks (NN) are a highly flexible class of models that draw inspiration from the structure of the brain. At a high level, neural networks try to mimic neural connectivity patterns and activation mechanisms using **nodes** with *activation values* and (directed) **edges** with *transmission weights*: Each node stores a numerical value thar represents its activation state (like a neuron firing or not firing), and each edge stores a weight representing the strength and direction (positive or negative) of signal transmission. When a signal propagates through the network, each node's value is scaled by and transmitted along its outgoing edges to connected nodes, who sum the incoming scaled values and apply a (non-linear) **activation function** to compute *their* activation state. Information starts at an **input layer** (where node values encode features of the data) and then propagates through the **hidden layers** to ultimately produce values at the **output layer** (where node values encode model predictions).



Convolutional neural networks (CNN) are a type of neural network that mimic how humans classify images by recognizing compositional features/patterns in the image that distinguish the classes (e.g., tigers have *ears*, cars have *wheels*). CNNs first identify low-level features in the image (small edges, patches of color, and so on) and then combines them to form higher-level features (ears, eyes, and so on). Eventually, the presence or absence of these higher-level features contributes to a higher or lower probability of any given class.



CNNs are able to build this feature hierarchy by using a combination of **convolution layers** and **pooling layers**. Convolution layers consist of **convolution filters**, which function as specialized templates designed to identify a particular local feature or pattern in an image (i.e., they're *feature detectors*). Convolution filters work by **convolving** the filter with a submatrix of the image (like a matrix dot product): if a submatrix resembles the filter, then its convolution with the filter will be large; otherwise, if the submatrix and filter are dissimilar, their convolution will be small. By sliding the filter window around the image and computing the convolutions of the region below, we obtain the **convolved image**, which highlights regions of the original image that match the convolution filter. The pooling layers then simply condense this image into a smaller summary image by calculating representative aggregates of non-overlapping chunks of the image. For example, the 2×2 max pooling operation summarizes each non-overlapping 2×2 block of pixels in an image using the maximum value in the block, which reduces the size of the image by a factor of 2 in each direction.



In summary, the architecture of a CNN typically alternates between convolution layers and pooling layers. The early layers serve to capture low-level features (e.g., edges), and the deeper layers build off of these low-level features to capture high-level features (e.g., an ear). Eventually, this information is flattened and fed into fully-connected layer(s) that perform higher-level reasoning on the extracted features to produce the model prediction.

Methodology

It turns out that there is a moderate class imbalance. In particular, there are 630 samples of the House Sparrow (most common) but only 37 samples of the Northern Flicker (least common). Since we only care about training our model to learn to distinguish the bird calls (and not bird *frequency* in the dataset), then we ought to balance the dataset so that the model isn't biased towards the more common birds. We can do so using *undersampling*, that is, just don't use all the samples of the more common birds. Specifically, we form our datasets so that the most common class has at most *twice* as many samples as the least common class. For the binary classification task (American Crow vs. Red-winged Blackbird), our dataset contained 66 samples of the American Crow and 132 samples of the Red-winged Blackbird.For the multi-class classification task, our dataset contained 37 samples of the Northern Flicker, 45 samples of the Black-capped Chickadee, 66 samples of the American Crow, and 74 samples of the remaining bird species.

In addition to undersampling, we also scale the data so that values are in the interval $[0, 1]$ (approximately). Since the values of the original data lie in the interval $[-80, 0]$ (approximately), we can do this by simply adding 80 and then dividing by 80. Scaling the data in

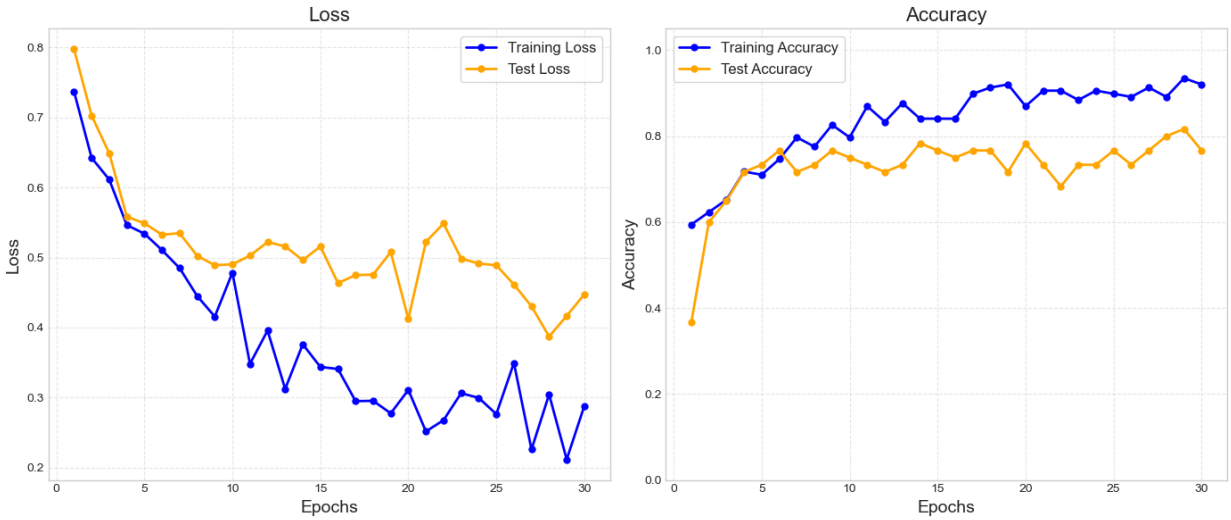
such a way should help models converge faster. (The reason why I say “approximately” is because there are a few positive data values that are *very* close to 0 so that, after being scaled, they are very slightly greater than 1.)

Now that we’ve talked about the data selection and preprocessing, let’s move on to model training and assessment. For each of the tasks (binary and multi-class classification), we split the dataset into a training set and a test set using a 70-30 split, and then train two models: a “lightweight” model (fewer layers/parameters) with less regularization, and “heavyweight” model (more layers/parameters) with more regularization (I *did* also fiddle around with the number of training epochs and batch size to find something reasonable). The two models were then compared on the basis of final model test accuracy (though we also consider the accuracy and loss plots to look for any signs of overfitting). The details of the model architecture can be found in the code.

Results

Binary Classification: American Crow vs. Red-winged Blackbird

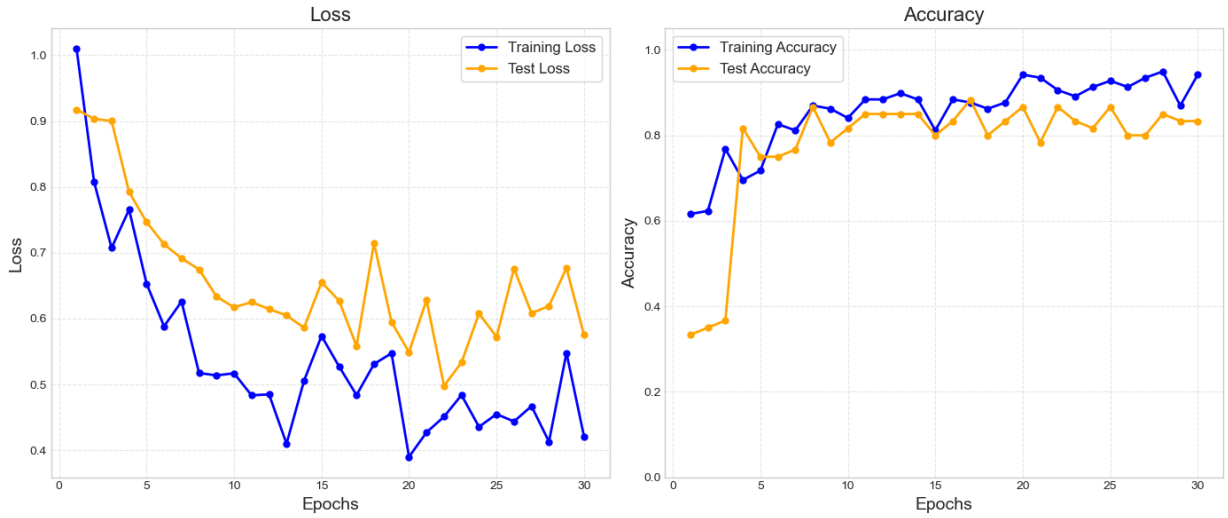
The lightweight model contains a single set convolutional and pooling layers that connect to a fully-connected hidden layer. Only batch normalization and dropout learning are used for regularization. The lightweight model trained for 30 epochs with a batch size of 4, taking a little more than 1 minute. The final test accuracy was 76.7%, and the loss and accuracy curves show a little volatility but no overfitting.



The confusion table doesn't highlight anything particularly problematic. If anything, it might show that the model is slightly biased towards the more common class (Red-winged Blackbirds), but this is very minor.

Truth	amecro	rewbla
Predicted		
amecro	11	5
rewbla	9	35

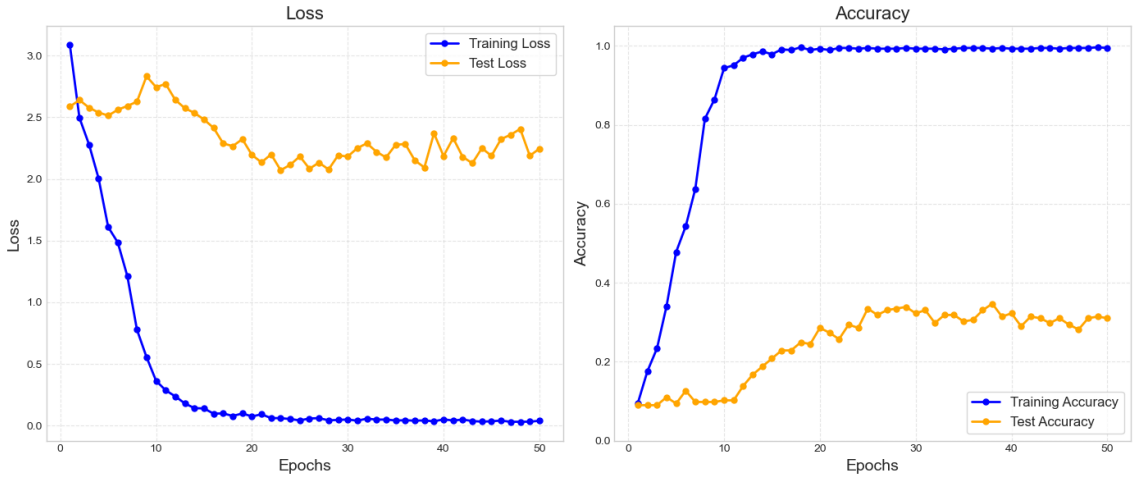
The heavyweight model contains two sets of convolution and pooling layers that then feed into a fully connected hidden layer. In addition to batch normalization and dropout learning, the heavyweight model also used weight regularization. The heavyweight model trained for 30 epochs with a batch size of 4, taking only 1 second longer than the lightweight model. The final test accuracy was 83.3% (a little better than the lightweight model), and the confusion table doesn't reveal any issues. Interestingly, the test accuracy much more lock-step with the training accuracy than seen with the lightweight model.



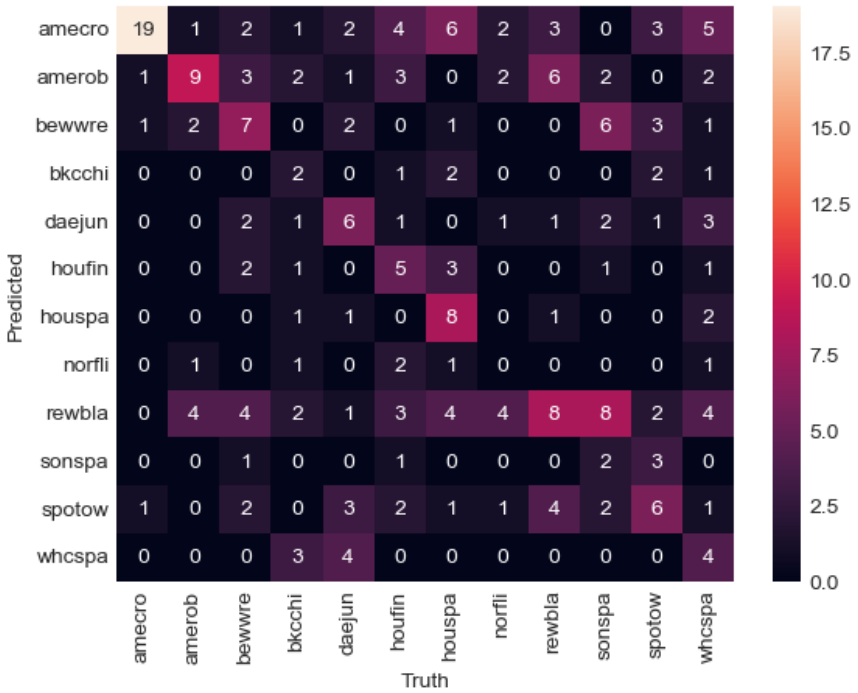
Truth	amecro	rewbla
Predicted		
amecro	14	4
rewbla	6	36

Multi-class Classification

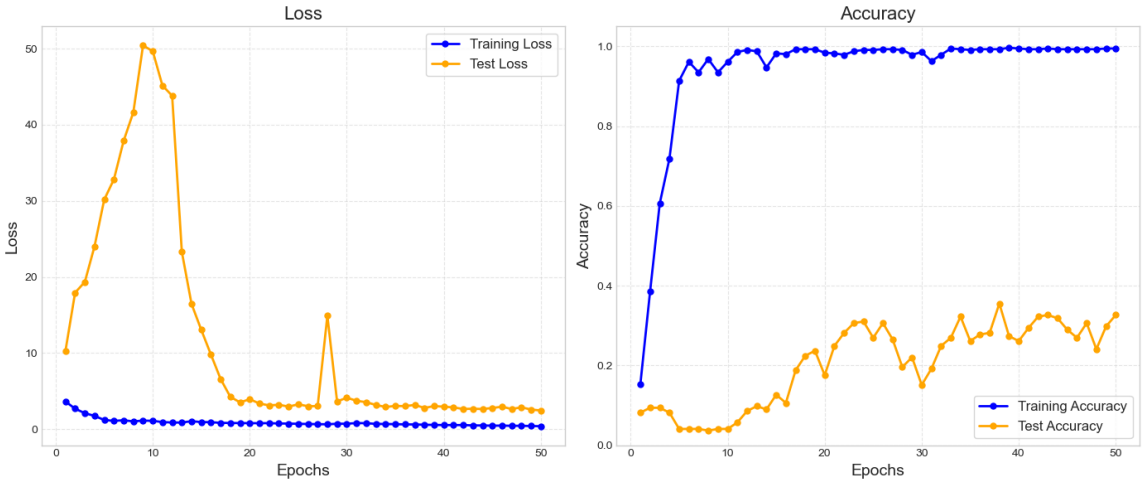
The lightweight model contains two sets of convolution and pooling layers that then feed into a fully connected hidden layer (same general structure as the binary heavyweight model, but with more parameters). The model trained for 50 epochs with a batch size of 16, taking almost 15 minutes. The final test accuracy was a measly 31%.



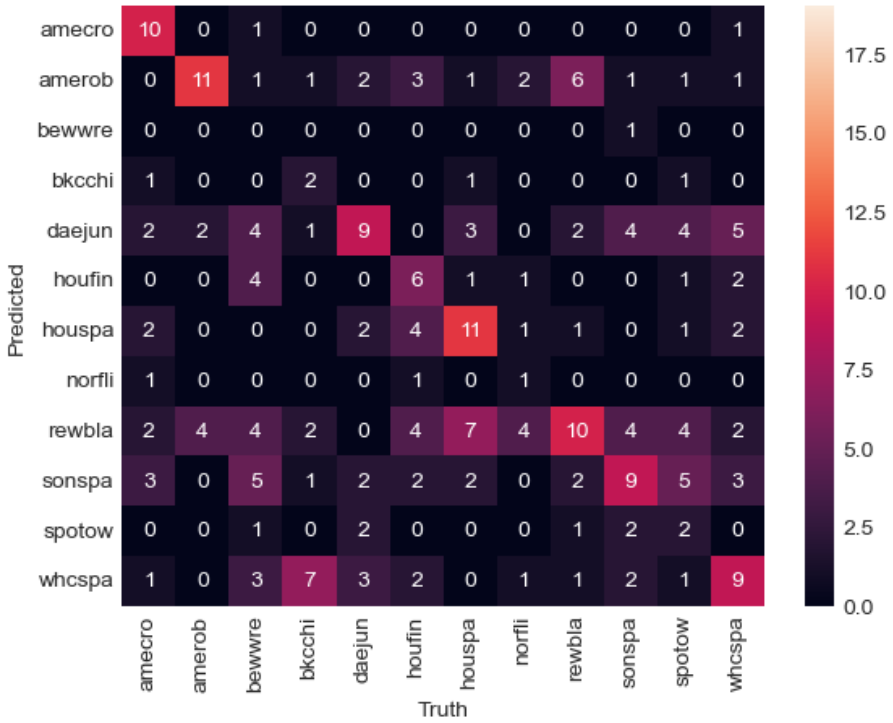
The lightweight model seems to most confuse Red-winged Blackbirds (rewbla) with Song Sparrows (sonspa).



The heavyweight model contains an extra set of convolution and pooling layers and just more parameters in general (e.g., more filters). Like with the binary models, the heavyweight model also introduces weight regularization. The model trained for 50 epochs with a batch size of 16, taking 30 minutes. The final test accuracy was only 32.7%.



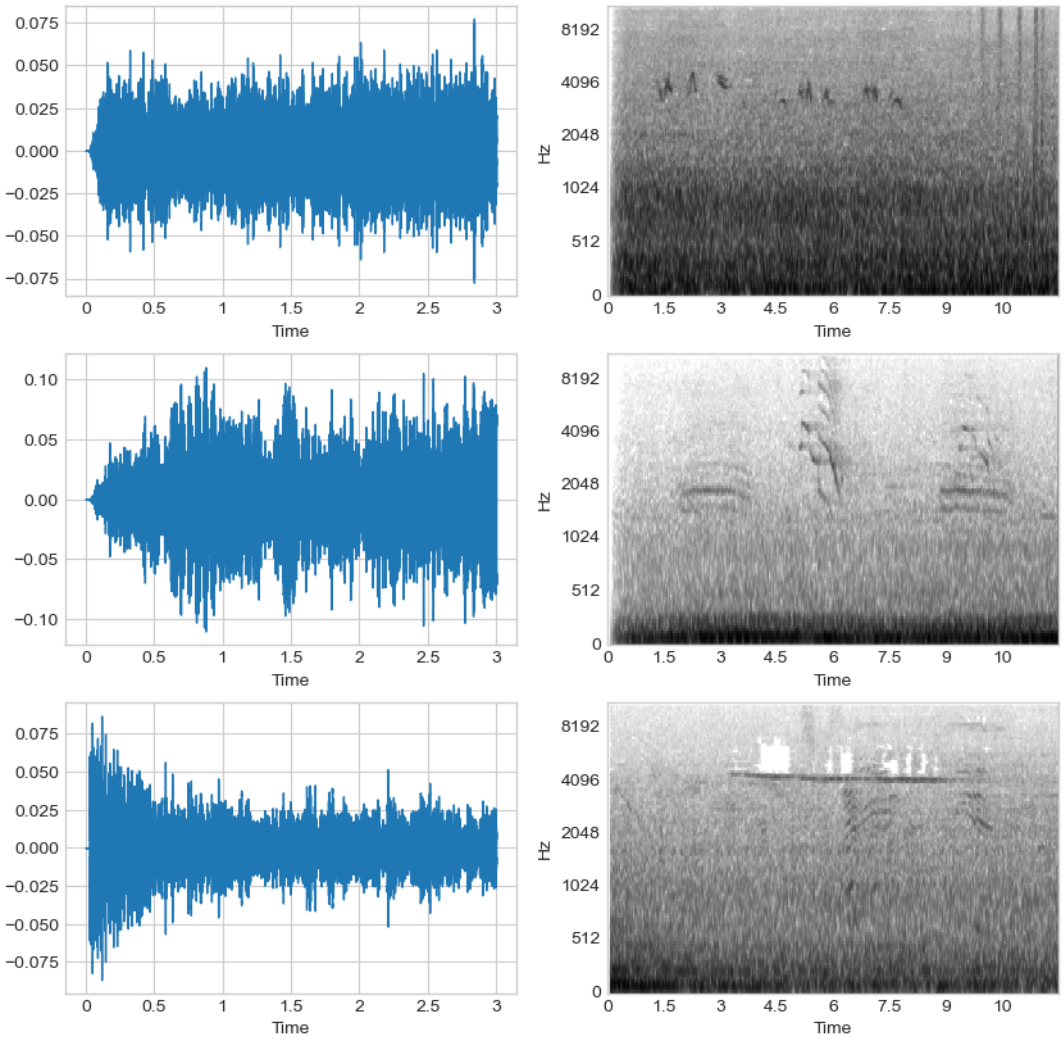
The heavyweight model seems to be better at distinguishing the Red-winged Blackbird and Song Sparrow, but is much worse at correctly identifying Bewick's Wrens (bewwre) and Spotted Towhees (spotow), specifically.



Overall, the multi-class classification models took much longer to train (there *is* more data) but produced disappointing results. The loss and accuracy plots are particularly striking because the training accuracy quickly goes to 100% while the test accuracy plateaus at just above 30% (no overfitting!). The only reasons I can think of that can explain this are (1) either the tested models just don't stack up to the task (the models are poor or the data is difficult to separate), or (2) the training set is not representative of the test set (which is *possible* with the number of samples we're working with, but unlikely).

External Test Data

The waveform and spectrographs of the three external test files are shown below.



Our heavyweight multi-class model predicts all three of the test files to be recordings of a White-crowned Sparrow (which is doubtful). If you look at the waveform plots and listen to the files, it becomes apparent that some of the files have background noise and even *multiple* bird calls! This could explain poor prediction results, though it is strange that the model predicted all three to be the same class. Unfortunately, I am no bird expert and cannot verify whether the predictions are true or not.

Conclusions

In this project, we built binary and multi-class classification CNN models to classify bird calls. The binary model, which distinguished between American Crows and Red-winged Blackbirds performed quite well, achieving 83.3% test accuracy. The multi-class classification model, which had to distinguish between all 12 species of birds, did not do as well, only achieving 32.7% test accuracy. Some possible reasons for this poor performance are (1) either the tested models just don't stack up to the task (the models are poor or the data is difficult to separate), or (2) the training set is not representative of the test set (which is *possible* with the number of samples we're working with, but unlikely).

The first possible reason raises the question, are CNNs really the best model for this task? As the task involves non-linear classification, some other candidate models are decision trees, SVMs, and KKN. Unfortunately, the data is very high-dimensional (the flattened spectrograms are length 66176) but with a low number of samples, and so these methods would likely require dimensionality reduction and/or feature engineering. One of the advantages of neural networks is that they automatically learn features directly from the data, which reduces the need for manual feature engineering. Furthermore, CNNs are literally designed for computer vision tasks, and have very important properties such as *local connectivity* and *translation invariance*. This is all to say that CNNs are surely the best model for

this task. Getting better model performance is just a matter of finding the right architectural configuration and/or collecting more data (or using tricks like data augmentation).

References

1. Karpathy, A. (n.d.). A recipe for training neural networks. https://karpathy.github.io/2019/04/25/recipe/

2. Wikimedia Foundation. (2025, May 8). *Convolutional Neural Network*. Wikipedia. https://en.wikipedia.org/wiki/Convolutional_neural_network